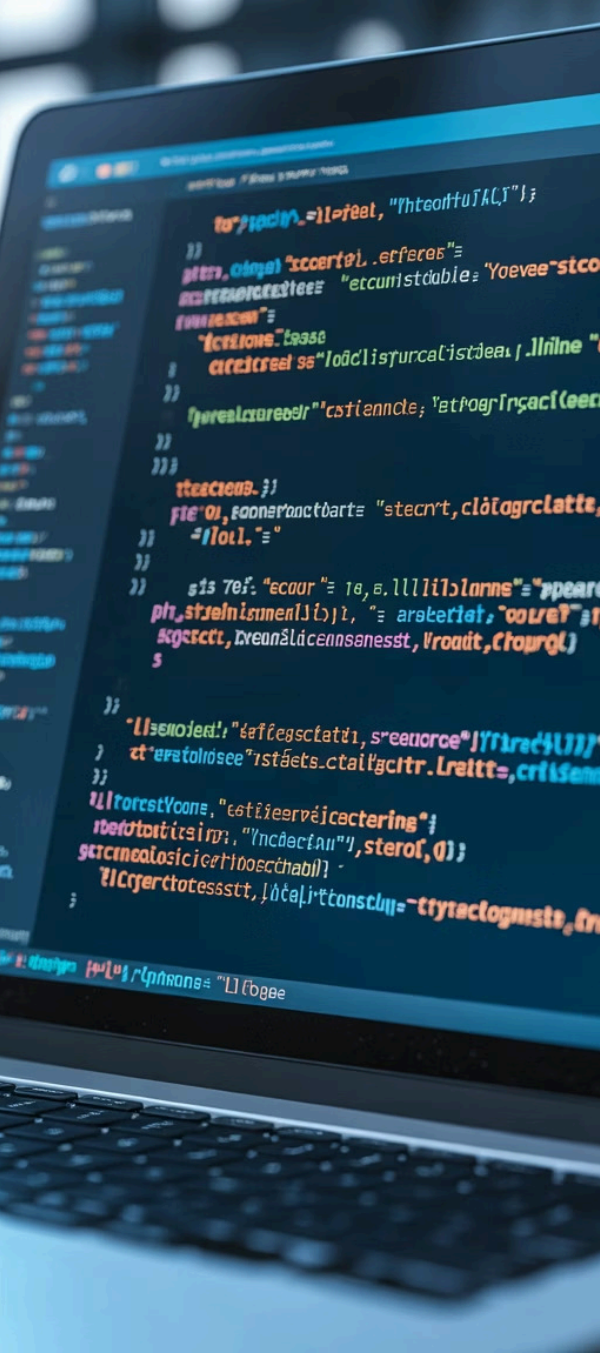




Reusabilidade do Código em PHP

Herança Múltipla com Interfaces e Traits

UNIVERSIDADE DO OESTE DE SANTA CATARINA – UNOESC

Ciência da Computação – Programação II

Prof.: Leandro Otavio Cordova Vieira



Objetivos da Aula



Reusabilidade de Código

Compreender os fundamentos e a importância da reutilização de código para criar sistemas mais eficientes e maintaináveis



Herança Múltipla

Explorar como interfaces e traits permitem herança múltipla em PHP, superando limitações da linguagem



Abstração e Reuso

Aprender a combinar conceitos de abstração com técnicas de reuso para criar código mais flexível e escalável



Prática em Duplas

Desenvolver um estudo de caso completo aplicando os conceitos aprendidos em atividade colaborativa

Relembrando Conceitos Fundamentais

Herança em PHP

O PHP é uma linguagem orientada a objetos que suporta **herança simples**, permitindo que uma classe filha herde características de apenas uma superclasse por vez. Isso significa que você pode estender uma única classe usando a palavra-chave `extends`.

A **herança múltipla direta** — onde uma classe poderia herdar de várias classes simultaneamente — é **proibida** no PHP. Essa restrição existe para evitar problemas como o "diamante da morte", onde conflitos de métodos e propriedades podem surgir.

A Solução do PHP

Felizmente, o PHP oferece mecanismos poderosos para contornar essa limitação: **Interfaces** e **Traits**. Essas ferramentas permitem que você obtenha os benefícios da herança múltipla sem os problemas associados.

Interfaces definem contratos que as classes devem seguir, enquanto Traits permitem o reuso horizontal de código entre classes não relacionadas hierarquicamente.



O que é uma Interface?

Definição de Contrato

Uma interface define um **contrato formal** que especifica quais métodos uma classe deve implementar. É como um acordo que garante que determinadas funcionalidades estarão disponíveis.

Apenas Assinaturas

Interfaces contêm **somente assinaturas de métodos** — isto é, o nome do método, seus parâmetros e tipo de retorno. Não há implementação, apenas a promessa de que existirão.

Padronização e Polimorfismo

Ao usar interfaces, você promove **padronização** no seu código e habilita o **polimorfismo**, permitindo que objetos de diferentes classes sejam tratados de forma uniforme através de uma interface comum.

- ❏ **Lembre-se:** Uma classe pode implementar múltiplas interfaces simultaneamente, o que torna as interfaces fundamentais para alcançar herança múltipla em PHP.

Exemplo Prático de Interface

```
interface Autenticavel {  
    public function login();  
    public function logout();  
}
```

Características Importantes

- **Nenhum corpo de método:** Apenas a declaração está presente, sem implementação
- **Contrato obrigatório:** Todas as classes que implementam esta interface devem definir os métodos `login()` e `logout()`
- **Visibilidade pública:** Métodos de interface são sempre públicos por padrão
- **Flexibilidade de implementação:** Cada classe pode implementar esses métodos da forma que fizer mais sentido para seu contexto



Esta interface Autenticavel garante que qualquer sistema de autenticação terá, no mínimo, as operações de login e logout disponíveis.

O que é uma Trait?

↔ Reuso Horizontal

Traits proporcionam **reuso horizontal de código**, permitindo compartilhar funcionalidades entre classes que não têm relação hierárquica. É diferente da herança vertical tradicional.

</> Métodos e Propriedades

Diferente das interfaces, traits podem conter **métodos completos com implementação** e até mesmo **propriedades**. Você está compartilhando código real, não apenas contratos.



Uso em Múltiplas Classes

Uma única trait pode ser **usada em várias classes diferentes**, promovendo o princípio DRY (Don't Repeat Yourself) e facilitando a manutenção do código.



Exemplo Prático de Trait

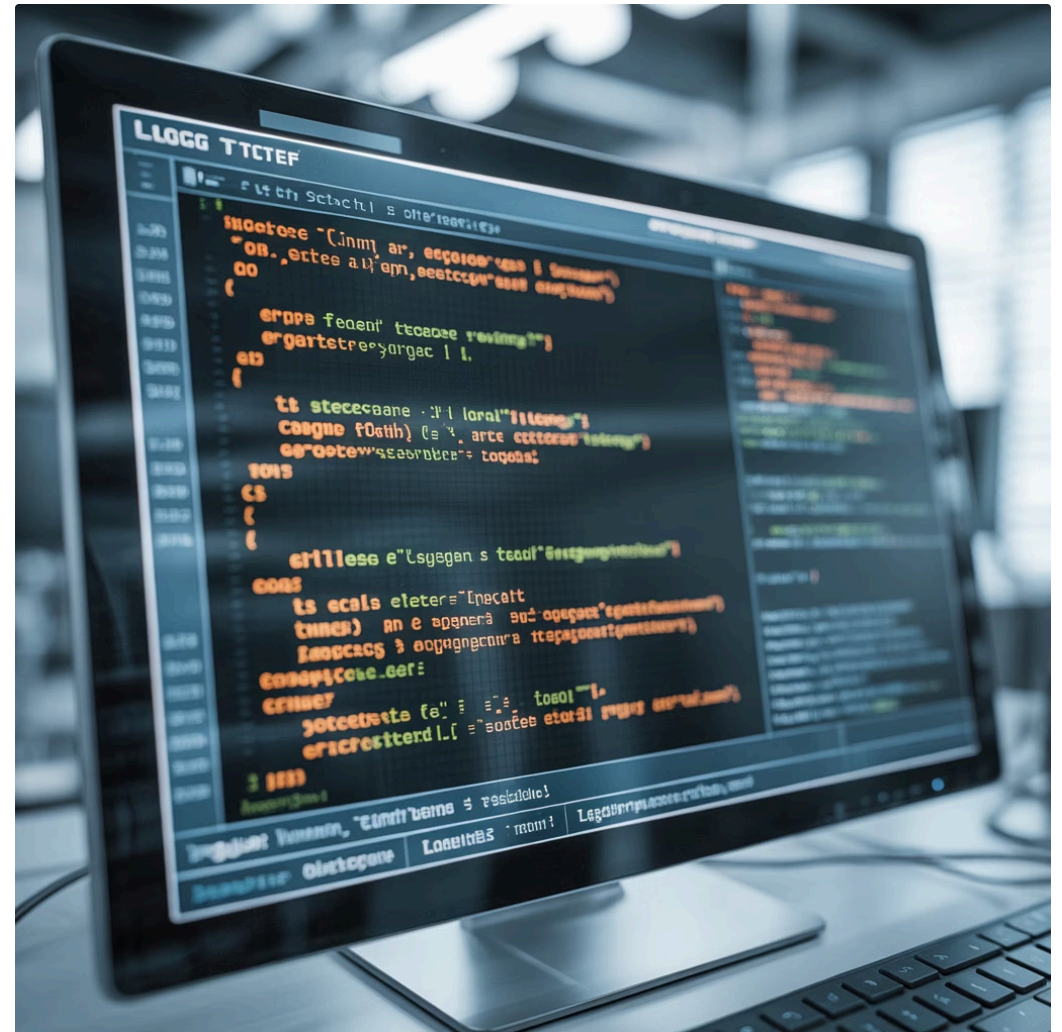
```
trait Logger {  
    public function registrarLog($msg) {  
        echo "[LOG] $msg\n";  
    }  
}
```

Como Usar

Para utilizar esta trait em qualquer classe, basta incluir a declaração `use Logger;` dentro do corpo da classe. A partir desse momento, o método `registrarLog()` estará disponível.

Vantagens

- Implementação completa e reutilizável
- Não precisa reescrever o código de logging em cada classe
- Facilita atualizações centralizadas
- Pode ser combinada com outras traits



Combinando Interface + Trait

A verdadeira força do PHP surge quando combinamos interfaces e traits. A interface define **o que** deve ser feito, enquanto a trait fornece **como** fazer.

Interface: O Contrato

```
interface Exportavel {  
    public function exportar();  
}
```

Define que classes exportáveis devem ter um método `exportar()`.

Trait: A Implementação

```
trait JsonExport {  
    public function exportar() {  
        return json_encode(  
            get_object_vars($this)  
        );  
    }  
}
```

Fornece uma implementação padrão para exportação em formato JSON.



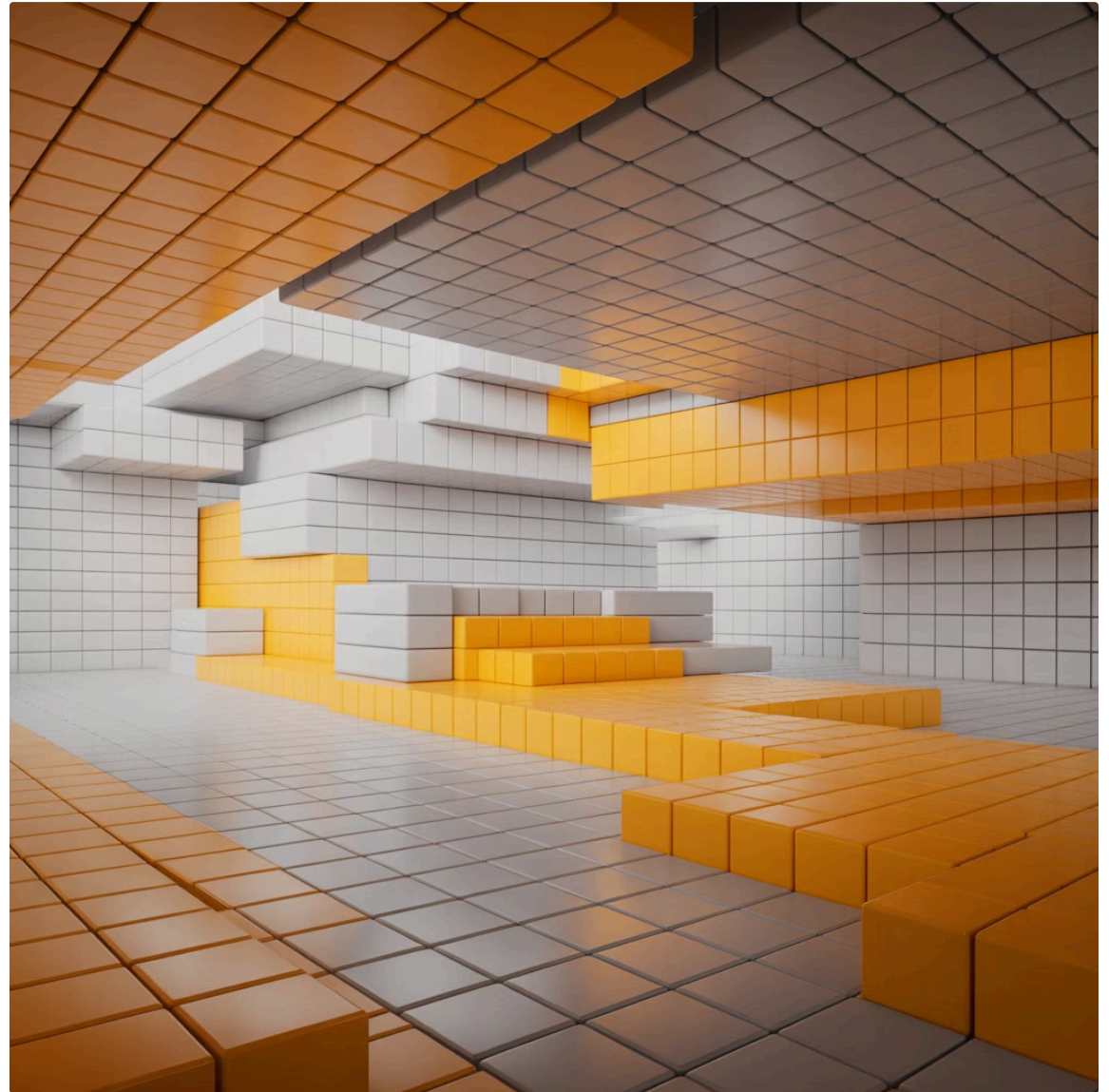
Classe que Usa Ambos

```
class Relatorio implements Exportavel {  
    use JsonExport;  
}
```

O Poder da Combinação

Esta simples declaração demonstra uma das características mais poderosas do PHP moderno:

- **Contrato garantido:** Implementa a interface `Exportavel`
- **Código reutilizado:** Usa a trait `JsonExport`
- **Zero duplicação:** Não precisamos reescrever a lógica de exportação



📌 **Resultado:** A classe `Relatorio` automaticamente ganha o método `exportar()` já implementado, cumprindo o contrato da interface sem escrever código adicional!

Comparativo: Interface vs Trait

Aspecto	Interface	Trait
Função	Define um contrato	Promove reuso de código
Métodos	Sem corpo (apenas assinatura)	Com corpo (implementação completa)
Herança	Múltipla (várias interfaces)	Múltipla (várias traits)
Ideal para	Polimorfismo e padronização	Compartilhar implementações comuns
Propriedades	Não suporta	Suporta propriedades
Quando usar	Quando precisa garantir que uma classe tenha certos métodos	Quando precisa compartilhar código entre classes não relacionadas

Resolvendo Conflitos entre Traits

Quando duas traits definem métodos com o mesmo nome, surge um conflito. O PHP exige que você resolva explicitamente qual versão usar.

O Problema

```
trait A {  
    function log() {  
        echo "A";  
    }  
}  
  
trait B {  
    function log() {  
        echo "B";  
    }  
}
```

Ambas as traits têm um método `log()`. Qual devemos usar?

A Solução

```
class Teste {  
    use A, B {  
        B::log insteadof A;  
    }  
}
```

A palavra-chave `insteadof` resolve o conflito, especificando que o método `log()` da trait B deve ser usado **no lugar** do método da trait A.

 **Dica avançada:** Você também pode usar `as` para criar aliases e manter ambos os métodos disponíveis com nomes diferentes!

Benefícios da Herança Múltipla via Traits

Reuso sem Duplicação

Escreva o código uma vez e use em múltiplas classes sem copiar e colar. Isso reduz drasticamente a duplicação e facilita correções de bugs.

Separação de Responsabilidades

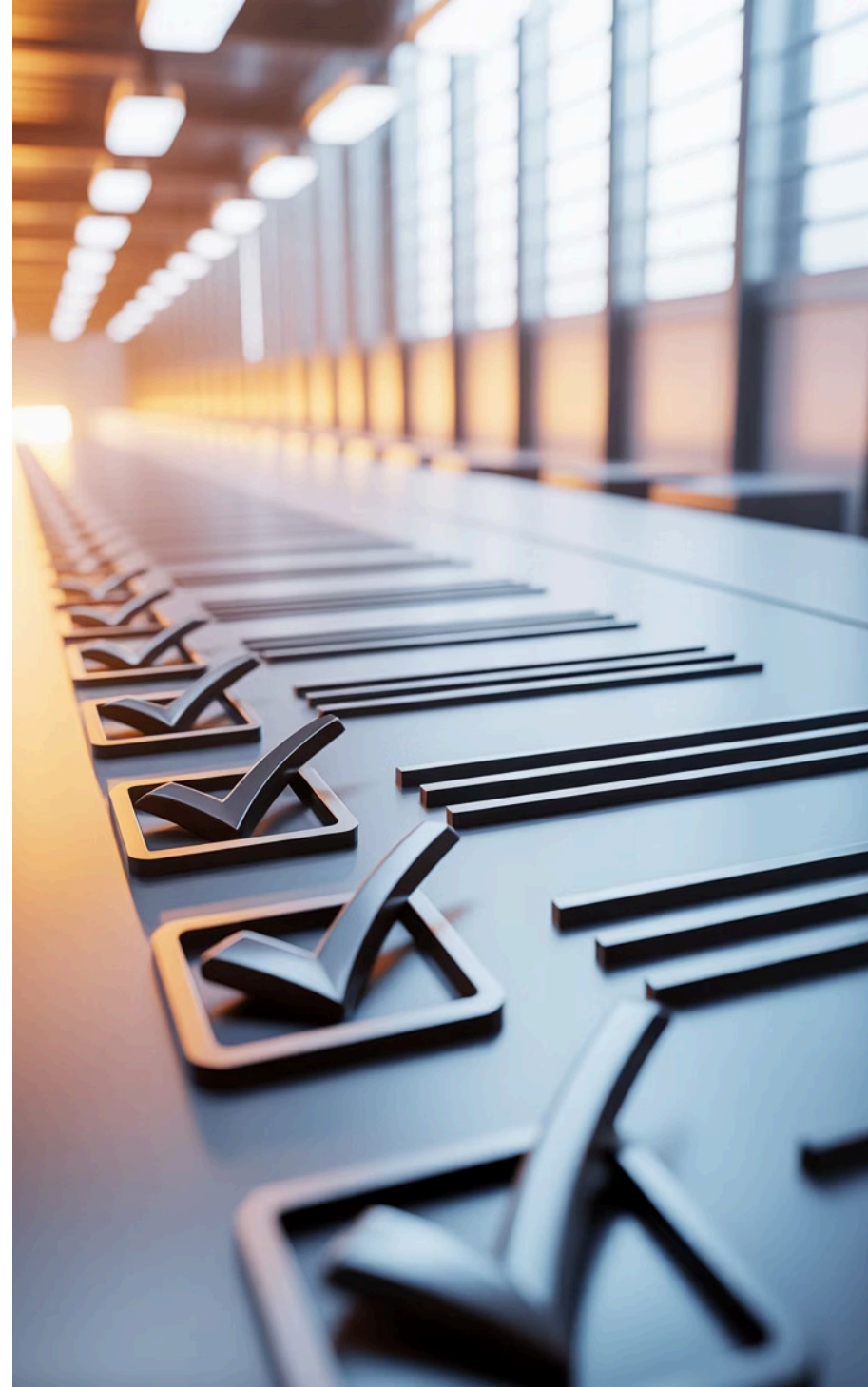
Cada trait pode encapsular uma responsabilidade específica (logging, validação, exportação), seguindo o princípio da responsabilidade única.

Manutenção Facilitada

Quando você precisa alterar uma funcionalidade, basta modificar a trait uma vez. Todas as classes que a utilizam automaticamente herdam a mudança.

Testes Simplificados

Traits podem ser testadas isoladamente, garantindo que o comportamento compartilhado funcione corretamente antes de ser usado em múltiplas classes.



Boas Práticas no Uso de Interfaces e Traits



1

Use Interfaces para Comportamentos

Sempre que você quiser garantir que uma classe tenha determinados métodos, use uma interface. Isso promove contratos claros e polimorfismo.

2

Use Traits para Implementações Compartilhadas

Quando várias classes precisam do mesmo código, encapsule em uma trait. Perfeito para funcionalidades transversais como logging, validação, ou formatação.

3

Evite Sobreposição Excessiva

Não crie traits ou interfaces demais com métodos que se sobrepõem. Mantenha cada trait focada em uma responsabilidade específica para evitar conflitos e confusão.

4

Documente Dependências

Se uma trait depende de métodos ou propriedades específicas da classe hospedeira, documente claramente essas expectativas para evitar erros em tempo de execução.



Estudo de Caso Progressivo

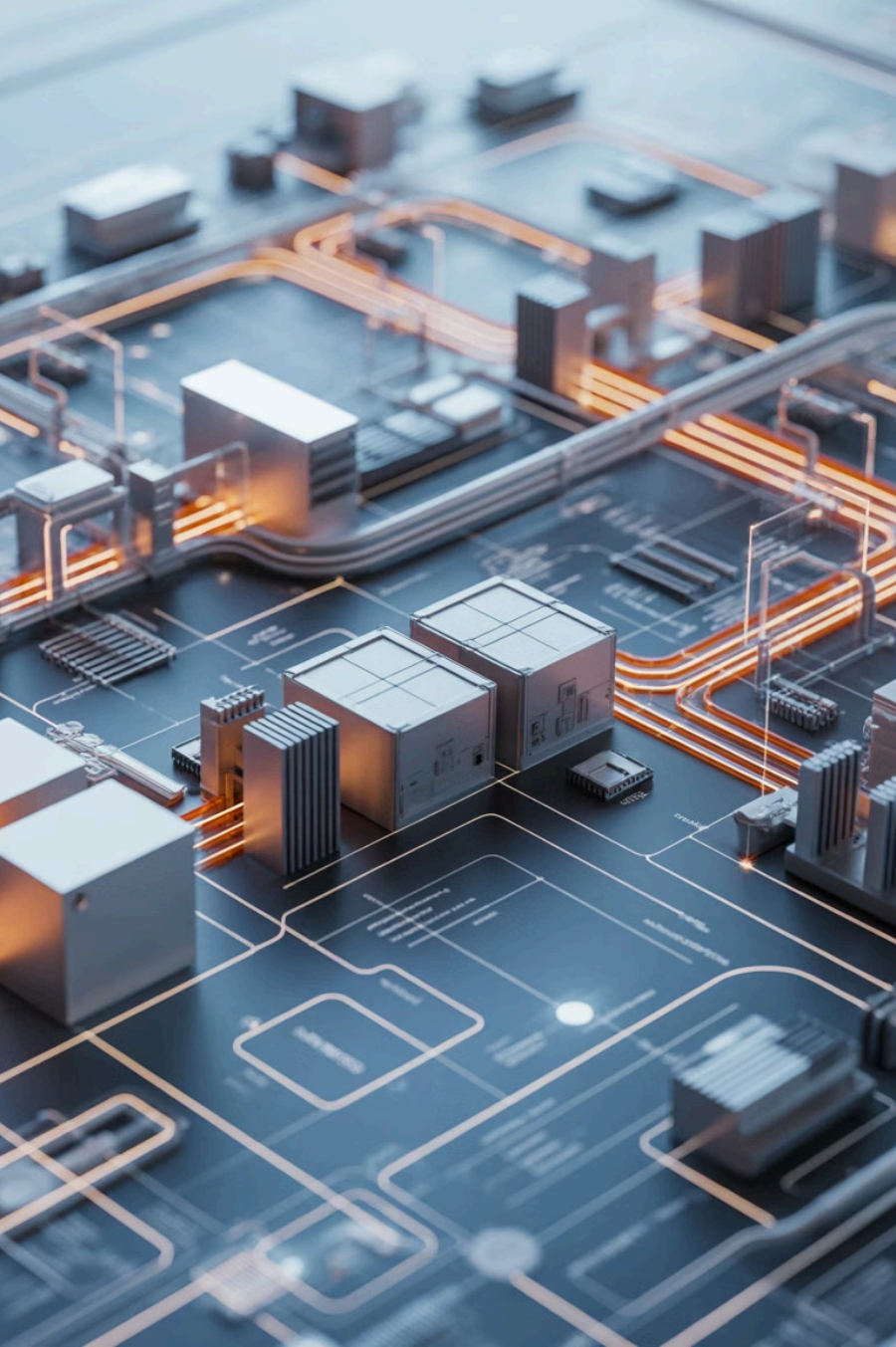
Sistema de Gestão de Usuários e Relatórios

Objetivo

Aplicar na prática os conceitos de interfaces e traits desenvolvendo um sistema completo de gestão. Este exercício será realizado **em duplas**, promovendo colaboração e troca de conhecimentos.

Estrutura

O estudo de caso está dividido em **três etapas progressivas**, cada uma construindo sobre a anterior. Vocês irão modelar, implementar e estender o sistema gradualmente.



Etapa 1 – Modelagem do Sistema

Nesta primeira etapa, vocês irão criar a fundação do sistema definindo os contratos e comportamentos reutilizáveis.

01

Interface Autenticavel

Crie uma interface com os métodos `login()` e `logout()` que definirá o contrato para qualquer entidade que precise de autenticação no sistema.

02

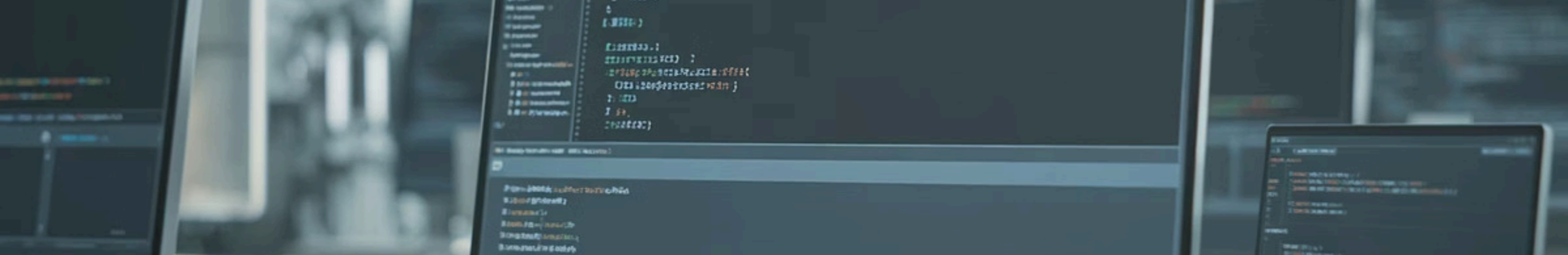
Trait Logger

Desenvolva uma trait com o método `registrarLog($mensagem)` que permitirá registrar eventos importantes do sistema de forma padronizada.

03

Trait Auditoria

Implemente uma trait com o método `registrarAcao($usuario, $acao)` para rastrear ações específicas realizadas por usuários administrativos.



Etapa 2 – Implementação das Classes

Agora vamos criar as classes concretas que utilizam as interfaces e traits definidas anteriormente.

1

Classe Usuario

Deve implementar a interface `Autenticavel` e usar a trait `Logger`. Os métodos `login()` e `logout()` devem registrar logs automaticamente.

2

Classe Administrador

Herda de `Usuario` (herança vertical) e adiciona a trait `Auditoria`. Cada ação administrativa deve gerar tanto logs quanto registros de auditoria.

3

Registros Automáticos

Configure o sistema para que logs e auditorias sejam gerados automaticamente sempre que os métodos relevantes forem chamados, sem código duplicado.

Etapa 3 – Extensão com Exportação

Na etapa final, vamos adicionar funcionalidade de exportação de dados ao sistema, aplicando novamente o padrão de interface + trait.

- Interface Exportavel

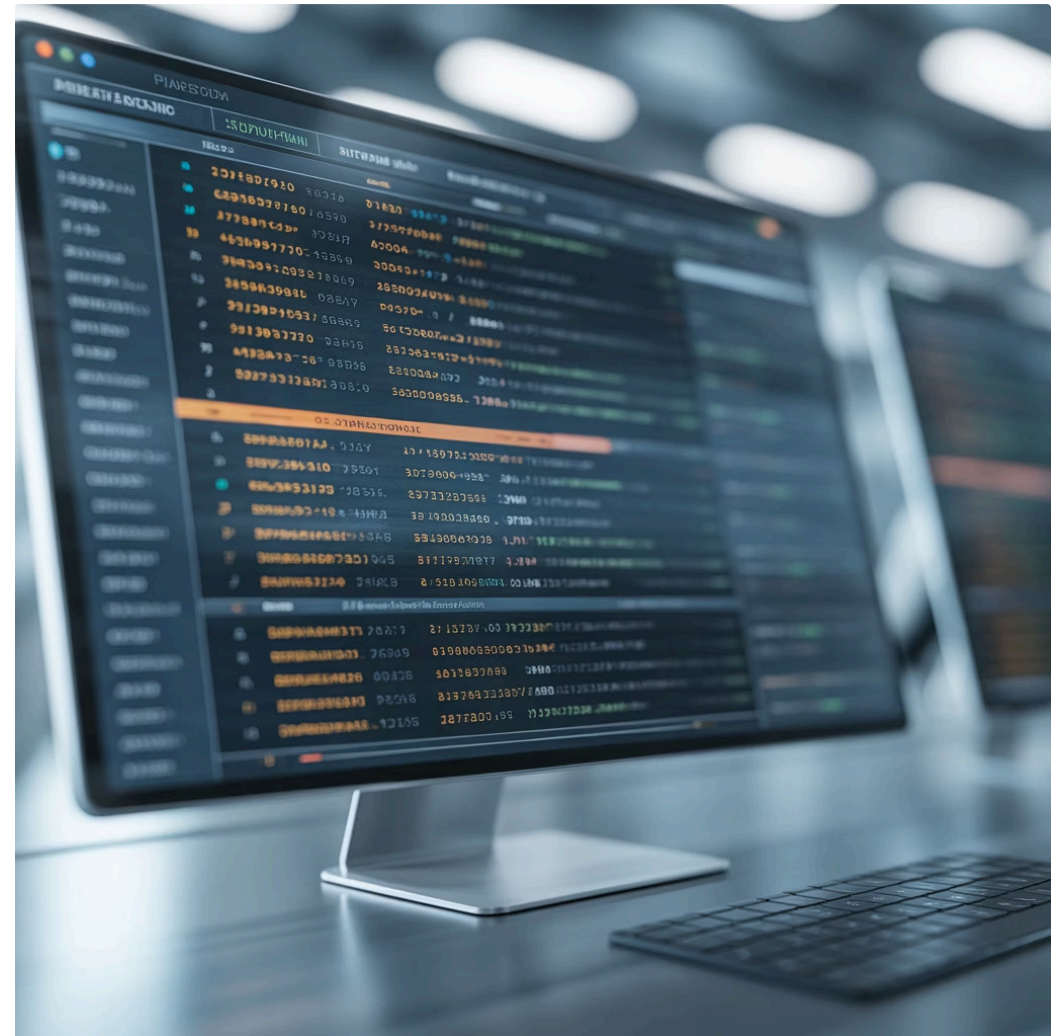
Crie com o método `exportar()`

- Classe Relatorio

Implementa `Exportavel` e usa `Logger`

- Exportação com Log

Gere exportações JSON registrando cada operação



📌 **Desafio extra:** Implemente também uma trait `CsvExport` e permita que o usuário escolha o formato de exportação!



Discussão em Duplas e Reflexão

1

Estratégias de Reuso

Comparem as diferentes abordagens utilizadas por cada dupla. Discutam quais traits foram criadas, como foram combinadas e quais benefícios cada estratégia trouxe.

2

Vantagens e Conflitos

Identifiquem as principais vantagens do uso de interfaces e traits no projeto. Houve algum conflito entre traits? Como foi resolvido? O que aprenderam com isso?

3

Princípios SOLID

Relacionem a solução desenvolvida com os princípios SOLID, especialmente o Princípio da Responsabilidade Única (SRP) e o Princípio da Segregação de Interface (ISP).

Preparem-se para compartilhar suas conclusões com a turma!